proj-init

Developer CLI Tool Blueprint

Scaffold any project stack in 3 seconds.

Production / Industrial Grade

Team

Abhi + Suhrit + Akshay

Timeline: 3-4 weeks

Language: Go

Phase 1: Abhi + Suhrit (Days 1-5)

Phase 2: Akshay joins

Built for developers who ship fast.

proj-init — Developer CLI Tool

> *Team:** Abhi + Suhrit + Akshay
>
> *Timeline:** 3–4 weeks (first 2–5 days: just Abhi + Suhrit, Akshay joins later)
>
> *Language:** Go
>
> *Public name:** `proj-init`
>
> *Tagline:** Scaffold any project stack in 3 seconds.
>
> *Grade:** Production / Industrial — not a learning project. Ships to real users.

Quality Standards (Industrial Grade)

This is not a tutorial project. Everything below is expected from Day 1:

Phas

Akshay not here yet. No waiting. Build the core.
A single CLI command that scaffolds a complete, working project stack in 3 seconds.

Inte

Direct mode — pick a template

List available templates

Version info

```
proj-init init


proj-init tauri-llm
proj-init whatsapp-bot
proj-init expense-splitter


proj-init list


proj-init --version
```

What the user gets: A ready-to-code folder with src/, CI/CD, Dockerfile, README with badges, .gitignore, LICENSE — everything compiles and runs first time.

Why

The flex: "I built a CLI tool that other developers use" = top 1% hiring signal.

Tech Stack

Why Go Over Rust

Verdict: Go lets 3 friends ship a working tool in 3-4 weeks instead of spending half that time learning Rust. Ship first, rewrite in Rust later if needed.

Proj

```
proj-init/
|
|-- main.go                    # Entry point
|-- go.mod / go.sum            # Go module
|
|-- cmd/
|   |-- root.go                # Root command (cobra)
|   |-- init.go                # proj-init init — interactive mode
|   |-- list.go                # proj-init list — show templates
|   `-- version.go             # proj-init --version
|
|-- internal/
|   |-- generator/
|   |   |-- generator.go       # Template engine wrapper
|   |   `-- vars.go            # Variable resolution
|   |-- templates/
|   |   |-- embed.go           # Embedded template filesystem
|   |   |-- tauri-llm/         # Template files for Tauri + LLM
|   |   |-- whatsapp-bot/      # Template files for WhatsApp bot
|   |   |-- expense-splitter/  # Template files for Flutter expense app
|   |   `-- ...                # More templates
|   |-- config/
|   |   `-- config.go          # Config file management
|   |-- output/
|   |   `-- output.go          # Colored output, spinners
|   `-- errors/
|       `-- errors.go          # Custom error types
|
|-- scripts/
|   |-- install.sh             # Unix install script
|   `-- install.ps1            # Windows install script
|
|-- .github/workflows/
|   |-- build.yml              # CI — build & test on push
|   `-- release.yml            # CD — release on tag via goreleaser
|
|-- .goreleaser.yaml           # Goreleaser config
|-- README.md                  # Main docs
|-- CONTRIBUTING.md            # Contributor guide
|-- LICENSE                    # MIT / Apache 2.0
`-- .gitignore
```

The 3 Equal Work Sectors

Every sector has ~equal complexity, ~equal LoC, ~equal difficulty. Nobody gets the easy part.

Secto

Who builds it: One team member (you or Suhrit)

Secto

Who builds it: The other early member (you or Suhrit)

Sector — Distribution & CI/CD

Who builds it: Akshay (joins later, but this sector is as meaty as the other two)

The

```
Sector  (CLI Engine)
|  Parses flags, collects variables
|  Calls Sector  with template name + variables
v
Sector  (Generator)
|  Reads embedded templates
|  Substitutes variables
|  Writes output to target directory
|  Returns path or error to Sector
v
Sector  displays result to user

Sector  (CI/CD)
|  Builds the binary from Sector  +  combined
|  Cross-compiles & publishes
|  Completely independent — no code coupling
```

No one blocks anyone. Each sector has clear boundaries.

Spri

Days 1-2 — Foundation (Abhi + Suhrit)

Days 3-5 — Core Features (Abhi + Suhrit)

Week 2 — Akshay Joins, Polish Core

Week 3 — Distribution & Quality

Week 4 — Shipping & Beyond

Template Roadmap (Stacks to Ship)

Success Criteria

- proj-init list shows available templates with descriptions
- proj-init tauri-llm creates a complete, compilable project in < 3 seconds
- Auto-generates README with badges, LICENSE, .gitignore, CI/CD
- Cross-compiles for Windows, macOS, Linux (amd64 + arm64)
- Published on at least 2 package managers (Homebrew + Scoop)
- All 3 team members contributed meaningful code
- Each sector has equal LoC and complexity (±20%)

Open Qu

- Project name — finalize proj-init or something else?

License — MIT or Apache 2.0?

GitHub org or personal account?

First template to ship — prioritize ORION stack or something simpler?

Do we dogfood immediately? — use proj-init to scaffold its own next version?

> ***Next action:** Pick Go, create GitHub repo, scaffold the skeleton. Let's go.*